

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

PROGRAMACION ORIENTADA A OBJETOS

APRENDIZAJE AUTONOMO 2

DESARROLLO DEL SISTEMA DE GESTION DE E-COMMERCE

Dato	Informacion
Elaborado por	Matias Castro Guerra
Docente guia	Palacios Morocho Milton Ricardo
Asignatura	Programacion Orientada a Objetos
Proyecto	Sistema de Gestion de E-commerce
Repositorio	https://github.com/MatixProx/sistema-gestion-ecommerce-poo.git
Fecha	Junio de 2026

1. Introduccion

El presente documento muestra el avance del Sistema de Gestion de E-commerce desarrollado para el Aprendizaje Autonomo 2. La entrega continua la planeacion realizada en el Aprendizaje Autonomo 1 y transforma los modulos definidos en una primera version funcional de codigo. El sistema se centra en el flujo principal de una tienda en linea: usuarios, productos, inventario, carrito, pedidos, pagos y reportes administrativos.

La propuesta mantiene relacion directa con Programacion Orientada a Objetos porque organiza el problema en tipos, atributos privados, metodos, composicion e interfaces. Tambien aplica estructuras de datos para resolver necesidades reales: mapas para buscar productos por ID, slices para manejar colecciones dinamicas y mapas de conteo para reportes.

2. Objetivo del avance

Implementar un avance significativo del Sistema de Gestion de E-commerce aplicando estructuras de datos, encapsulacion, manejo de errores e interfaces, de forma coherente con la planeacion previa y con las funcionalidades definidas en la primera entrega.

Objetivos especificos

1. Continuar el plan del Aprendizaje Autonomo 1 mediante una primera version codificada del sistema.
2. Identificar los tipos o clases desarrolladas y relacionarlas con los modulos del e-commerce.
3. Implementar encapsulacion para proteger datos internos como correo, precio, stock, estado e items del carrito.
4. Agregar manejo de errores para validar campos, cantidades, stock, estados y pagos.
5. Usar interfaces para representar metodos de pago con comportamiento polimorfo.
6. Comentar las funcionalidades mas complejas para que el codigo sea entendible y mantenible.
7. Preparar evidencia de ejecucion, pruebas y guion para los videos solicitados.

3. Continuacion del plan anterior

En la primera entrega se planifico un sistema de e-commerce con modulos de usuarios, productos, carrito, pedidos, pagos, inventario y reportes. Para esta segunda entrega se priorizo una version de consola que demuestra el flujo de compra sin depender todavia de interfaz grafica, base de datos o pasarela de pago real. Esta decision permite evidenciar primero la logica de negocio, que es la base del sistema.

Modulo planificado en AA1	Avance implementado en AA2	Estado
Gestion de usuarios	Tipos Usuario, Cliente y Administrador con validaciones de datos y roles.	Implementado como base funcional
Gestion de productos	Tipo Producto con categoria, precio, stock, descripcion y metodos de actualizacion.	Implementado
Inventario	Inventario con map[string]*Producto para agregar, buscar, listar y detectar bajo stock.	Implementado
Carrito de compras	Carrito con slice de DetalleCarrito para agregar, eliminar, cambiar cantidades y calcular total.	Implementado
Gestion de pedidos	Pedido con estados controlados y reduccion de stock al confirmar la compra.	Implementado
Gestion de pagos	Interfaz MetodoPago con tarjeta, transferencia y contra entrega.	Implementado
Reportes administrativos	ReporteService para ventas, pedidos por estado, productos vendidos y bajo stock.	Implementado

4. Fuentes revisadas y evaluacion critica

Para fundamentar el avance se revisaron fuentes de tipo academico, tecnico y del propio proyecto. No se tomaron como verdades absolutas: se compararon con las necesidades reales del sistema y con el alcance de la actividad.

Fuente o punto de vista	Aporte para el proyecto	Cuestionamiento o limite reconocido
Plan del Aprendizaje Autonomo 1	Define modulos, clases, alcance y relacion con POO para el e-commerce.	Es una planeacion inicial; no prueba por si sola que el software funcione. Por eso ahora se codifica y se ejecutan pruebas.
Documentacion oficial de Go	Permite aplicar structs, metodos, paquetes, campos no exportados e interfaces.	Go no usa clases tradicionales como Java; por eso se explica que la equivalencia se trabaja con structs, metodos y composicion.
Criterio de POO visto en clase	Orienta el uso de encapsulacion, responsabilidades, interfaces y polimorfismo.	No basta con mencionar los principios; deben verse en codigo funcionando.
Necesidad empresarial de una tienda en linea	Justifica productos, inventario, pedidos, pagos y reportes.	La version actual simula el proceso y no integra bancos, base de datos real ni envios externos.

5. Posicion del estudiante

Mi posicion es que el avance cumple mejor la finalidad de la actividad si primero se construye una base de dominio clara y comprobable antes de agregar pantallas o conexiones externas. En un sistema de e-commerce, la logica de negocio debe quedar protegida: un producto no debe aceptar precio cero, un pedido no debe crearse con carrito vacio, y un pago no debe confirmar una compra invalida.

Tambien reconozco un limite: esta etapa todavia no representa un sistema comercial completo. Aun falta persistencia en base de datos, interfaz grafica y servicios externos. Sin embargo, el avance si demuestra la estructura principal del software y deja una base ordenada para continuar el desarrollo.

6. Tipos o clases desarrolladas

Tipo / struct	Responsabilidad	Concepto aplicado
Usuario	Guarda datos comunes de cualquier actor del sistema.	Encapsulacion, validaciones, getters y setters
Cliente	Representa al usuario que realiza compras.	Composicion de Usuario
Administrador	Representa al usuario que gestiona productos y reportes.	Composicion, rol y permisos
Categoria	Clasifica los productos.	Struct y validacion de campos
Producto	Representa el articulo vendido con precio y stock.	Encapsulacion, control de stock
Inventario	Administra productos mediante identificador.	Map como estructura de datos
Carrito	Maneja productos seleccionados y calcula total.	Slice, metodos y validacion de cantidades
Pedido	Formaliza la compra y controla estados.	Estados, detalles y reglas de negocio
Pago	Registra metodo, monto, estado y pedido asociado.	Struct y metodos
MetodoPago	Contrato comun para pagar con diferentes metodos.	Interface y polimorfismo
ReporteService	Genera indicadores administrativos.	Slices, maps, goroutines y canales

7. Encapsulacion implementada

La encapsulacion se implementa declarando campos privados dentro de los structs. Por ejemplo, Usuario no expone directamente id, nombre, correo, contrasena ni rol. Producto tampoco expone directamente

precio ni stock. Para leer datos se usan metodos como Nombre(), Precio() y Stock(); para modificar se usan setters que validan antes de guardar.

```
type Producto struct {
    id          string
    nombre      string
    precio      float64
    stock       int
    descripcion string
    categoria   Categoria
}

func (p *Producto) SetPrecio(precio float64) error {
    if precio <= 0 {
        return fmt.Errorf("precio %.2f: %w", precio, ErrPrecioInvalido)
    }
    p.precio = precio
    return nil
}
```

Con esta estructura se evita que otras partes del programa ingresen valores incorrectos. Esto mejora la seguridad logica del sistema y mantiene las reglas del negocio en un solo lugar.

8. Manejo de errores

El archivo errors.go centraliza errores reutilizables. Esto permite que el sistema responda de forma clara ante campos vacios, correo invalido, precio invalido, stock insuficiente, carrito vacio, pago invalido o estado de pedido no permitido.

```
var (
    ErrCampoVacio      = errors.New("el campo no puede estar vacio")
    ErrPrecioInvalido  = errors.New("el precio debe ser mayor a cero")
    ErrStockInsuficiente = errors.New("stock insuficiente para completar la operacion")
    ErrCarritoVacio    = errors.New("el carrito esta vacio")
    ErrPagoInvalido    = errors.New("el metodo de pago no pudo procesarse")
)
```

El uso de errores con contexto facilita encontrar el problema exacto. Por ejemplo, si un producto no tiene stock suficiente, el mensaje puede incluir el nombre del producto junto con el error general.

9. Interfaces y polimorfismo

La interfaz MetodoPago define un contrato comun para cualquier forma de pago. TarjetaPago, TransferenciaPago y ContraEntregaPago implementan esa interfaz, pero cada una procesa el pedido de forma diferente.

```
type MetodoPago interface {
    Nombre() string
    Procesar(pedido *Pedido) (*Pago, error)
}
```

Esta parte evidencia polimorfismo porque el sistema puede trabajar con una variable de tipo MetodoPago sin depender de una implementacion concreta. En el futuro se podria agregar PayPal, PayPhone u otra pasarela creando un nuevo struct con los mismos metodos.

10. Estructuras de datos aplicadas

Estructura de datos	Lugar donde se usa	Justificacion
map[string]*Producto	Inventario	Permite buscar productos rapidamente por ID.

[]DetalleCarrito	Carrito	Permite manejar varios productos y modificar cantidades.
[]Pedido	Reportes	Permite recorrer pedidos para calcular indicadores.
map[EstadoPedido]int	Reporte de estados	Agrupar pedidos por pendiente, pagado, enviado o cancelado.
map[string]int	Productos mas vendidos	Suma cantidades vendidas por nombre de producto.

11. Funcionalidades complejas comentadas

La funcion GenerarResumen esta comentada porque combina varias operaciones de reporte y usa concurrencia con goroutines y canales. Los comentarios explican por que se separan ventas, estados y productos vendidos en tareas diferentes.

```
// GenerarResumen usa goroutines y canales para calcular partes del reporte en paralelo.
// Se comenta porque es una funcionalidad mas compleja: separa ventas, estados y productos
// vendidos
// para que cada tarea se resuelva sin bloquear a las demas.
```

Estos comentarios ayudan a que otro integrante del grupo o el docente comprenda la parte mas compleja sin tener que adivinar la intencion del codigo.

12. Flujo de funcionamiento demostrado

8. Se crea una categoria de productos.
9. Se crean productos con precio y stock validado.
10. Se agregan productos al inventario.
11. Se registra un cliente con datos validados.
12. El cliente agrega productos al carrito.
13. Se crea un pedido y se reduce el stock.
14. Se procesa el pago con una implementacion de MetodoPago.
15. Se genera un reporte de ventas, estados, productos vendidos y bajo stock.

13. Pruebas y evidencia de ejecucion

Se agregaron pruebas unitarias para validar que la logica principal no dependa solo de una demostracion manual.

Prueba	Que valida
TestProductoValidaPrecio	No permite crear productos con precio invalido.
TestCarritoCalculaTotal	Calcula correctamente el total del carrito.
TestPedidoReduceStock	Reduce el stock al crear un pedido.
TestMetodoPagoPolimorfico	Procesa pagos mediante la interfaz MetodoPago.
TestReporteConcurrente	Genera reportes administrativos con el total esperado.

Comando de pruebas:
go test ./...

Resultado esperado:
ok github.com/MatixProx/sistema-gestion-ecommerce-poo/internal/ecommerce

Comando de demostracion:
go run ./cmd/demo

Salida esperada resumida:
Cliente: Matias Castro (cliente)
Total carrito: \$108.90

Pedido: PED-001 | Estado: pagado | Total: \$108.90

Pago: PAG-TAR-PED-001 | Metodo: tarjeta | Estado: confirmado

Ventas confirmadas: \$108.90

14. Cumplimiento de la rubrica

Criterio	Como se cumple para apuntar a Alcanzado
Evidencia Paso 1	Se relaciona el avance con el plan anterior, se usan fuentes tecnicas y se cuestionan sus limites.
Posicion del estudiante Paso 1	Se declara una postura clara: construir primero la logica de dominio antes de agregar pantallas o integraciones externas.
Conclusiones y logros Paso 1	Se explican logros concretos, consecuencias y limites de la etapa actual.
Encapsulacion Paso 2	Campos privados, getters y setters con validaciones en Usuario, Producto, Categoria y otros tipos.
Manejo de errores e interfaces Paso 2	Errores centralizados en errors.go e interfaz MetodoPago con varias implementaciones.
Comentarios en funcionalidades complejas	Comentarios en ReporteService.GenerarResumen y en partes clave del codigo para facilitar el entendimiento.

15. Guion sugerido para el video explicativo del avance

16. Presentar el proyecto como continuacion del Aprendizaje Autonomo 1.
17. Explicar que el sistema seleccionado es un e-commerce y que ahora ya existe codigo funcional.
18. Mostrar el README y la estructura del repositorio.
19. Explicar los tipos principales: Usuario, Cliente, Producto, Inventario, Carrito, Pedido, Pago y ReporteService.
20. Resaltar encapsulacion, manejo de errores e interfaces.
21. Cerrar indicando los limites actuales y los siguientes pasos: base de datos, interfaz grafica y pasarela real.

16. Guion sugerido para el video de demostracion

22. Abrir la terminal dentro del proyecto.
23. Ejecutar go test ./... y mostrar que las pruebas pasan.
24. Ejecutar go run ./cmd/demo.
25. Explicar la salida: cliente, total, pedido pagado, pago confirmado, ventas, pedidos por estado, productos vendidos y bajo stock.
26. Mostrar el archivo pago.go para explicar la interfaz MetodoPago.
27. Mostrar el archivo producto.go o usuario.go para explicar encapsulacion.
28. Mostrar el archivo reporte.go para explicar comentarios y funcionalidad compleja.

17. Conclusiones y logros relacionados

El avance logra convertir la planeacion del sistema en una base funcional de software. Ya no se presenta solamente una idea de e-commerce: se demuestra con codigo que existen productos, inventario, carrito, pedidos, pagos y reportes.

El logro mas importante es la organizacion de la logica de negocio mediante tipos con responsabilidades claras. Esto permite mantener el sistema entendible y facilita que en una siguiente etapa se agreguen base de datos, interfaz grafica o servicios externos.

Como consecuencia positiva, el proyecto queda preparado para crecer sin perder orden. La encapsulacion protege los datos, el manejo de errores evita operaciones incorrectas, y las interfaces permiten agregar nuevos metodos de pago sin modificar toda la estructura. El limite principal es que todavia es una version de consola y simula algunos procesos, por lo que no reemplaza aun a un e-commerce real completo.

18. Referencias bibliograficas

- The Go Authors. (s. f.). Effective Go. https://go.dev/doc/effective_go
- The Go Authors. (s. f.). The Go Programming Language Specification. <https://go.dev/ref/spec>
- GitHub Docs. (s. f.). About repositories. <https://docs.github.com/>
- Object Management Group. (s. f.). Unified Modeling Language. <https://www.uml.org/>
- Castro Guerra, M. (2026). Planeacion del software para un Sistema de Gestion de E-commerce. Aprendizaje Autonomo 1.